LoRA vs fine-tuning completo vs prompting para clasificación de sentimiento en tweets en español

Deep Learning — Proyecto Final · Universidad del Valle de Guatemala
Roberto Nájera, Jose López, Olivier Viau, Osman García
Octubre 2026

Repositorio: https://github.com/Ultimate-Truth-Seeker/ProyectoFinalDL

1. Descripción del problema

El análisis de sentimiento en redes sociales se usa en la industria (monitoreo de marca, atención al cliente) y en la academia (opinión pública). Adaptar un modelo de lenguaje a esta tarea tiene un costo: el *fine-tuning* completo actualiza cientos de millones de parámetros y exige memoria para gradientes y estados del optimizador, algo que no siempre está disponible (por ejemplo, una GPU T4 gratuita de Colab). Existen alternativas más baratas: usar un LLM sin entrenar (*prompting*) o entrenar solo un pequeño número de parámetros (LoRA).

Pregunta de investigación: ¿cuándo LoRA ofrece una calidad comparable al fine-tuning completo con una fracción del costo, y cuándo basta el prompting sin entrenamiento?

2. Análisis

- **Datos.** `cardiffnlp/tweet_sentiment_multilingual` (subconjunto español): tweets con 3 clases perfectamente balanceadas (negativo, neutro, positivo). La descarga real trae 1,839 / 324 / 870 ejemplos (train/validación/test), uno más por split que lo que indica la ficha. Los tweets son informales, con ironía, errores ortográficos, emojis y menciones (`@user`).
- **Test pequeño.** Con 870 ejemplos, diferencias de 1–2 puntos de F1 no son concluyentes, por lo que cada configuración principal se corrió con 5 semillas y se reporta media ± desviación estándar.
- **Riesgos.** Sobreajuste (1.8 k ejemplos para 278 M parámetros), olvido catastrófico, y la ambigüedad intrínseca de la clase «neutro».
- **Restricción de cómputo.** GPU T4 (15 GB, sin bf16), por lo que se usa precisión mixta fp16.
- **Segunda tarea.** `mteb/SpanishNewsClassification` (12 categorías, 2,048 noticias, un único split) con particiones propias estratificadas 70/10/20, para verificar si las conclusiones dependen del dataset.

3. Propuesta de solución

Se comparan tres estrategias bajo el mismo protocolo (mismos splits, semillas y métricas):

1. **Prompting sin entrenamiento** (zero-shot y few-shot con 3 ejemplos por clase) con `Qwen2.5-1.5B-Instruct`.
2. **Fine-tuning completo** de `xlm-roberta-base` (278 M parámetros).
3. **LoRA** sobre el mismo `xlm-roberta-base`.

Se complementa con una ablación de LoRA (rango r ∈ {2, 8, 16}; adaptar solo query/value vs. todas las capas lineales), la segunda tarea de noticias y un análisis de errores con SHAP. Métricas: accuracy, F1 macro, parámetros entrenables, memoria pico de GPU y tiempo.

Elección de modelos para la T4. XLM-R base es multilingüe, y su fine-tuning completo cabe holgadamente (≈5.4 GB de pico), lo que permite una comparación limpia con LoRA. Se descartó `cardiffnlp/twitter-xlm-roberta-base-sentiment` porque ya fue entrenado con estos tweets (fuga de datos). Para el prompting se necesita un modelo generativo: Qwen2.5-1.5B-Instruct ocupa ≈3.5 GB en fp16 y entiende español.

4. Descripción de la solución

Arquitectura. XLM-R es un Transformer *encoder* (self-attention multi-cabeza, S5–S6) preentrenado con *masked language modeling* en 100 idiomas. Se añade una cabeza de clasificación sobre el token `<s>` y se optimiza la entropía cruzada.

Entrenamiento (S8). AdamW, weight decay 0.01, *warmup* del 10 % seguido de decaimiento lineal, batch 32, 8 épocas, longitud máxima 128 tokens, fp16. Se conserva el checkpoint con mejor F1 macro de validación (*early stopping* por selección de época), que controla el sobreajuste. Tasa de aprendizaje: $2 \cdot 10^{-5}$ para fine-tuning completo y $4 \cdot 10^{-4}$ para LoRA (las tasas típicas de cada enfoque; no se hizo búsqueda de hiperparámetros).

LoRA (S9). Para cada matriz de pesos preentrenada $W \in \mathbb{R}^{d \times k}$ se congela W y se aprende una actualización de rango bajo: $W' = W + (\alpha/r) \cdot B \cdot A$, con $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$. Configuración principal: r = 8, α = 16, dropout 0.1, aplicado a las proyecciones *query* y *value*; la cabeza de clasificación se entrena completa.

Prompting (S13). Se da al LLM una instrucción en español (más 3 ejemplos por clase en few-shot) y la clase se elige comparando los logits del primer token de «negativo», «neutro» y «positivo». Esto evita respuestas fuera de formato.

Medición. Parámetros entrenables (`requires_grad`), memoria pico (`torch.cuda.max_memory_allocated`) y tiempo total de entrenamiento (incluye la evaluación de cada época). Para el prompting, el tiempo es de inferencia sobre validación + test.

Explicabilidad (S15). SHAP (*Partition explainer* con enmascaramiento de tokens) sobre 40 errores y 20 aciertos del modelo LoRA (semilla 0) en el conjunto de test.

5. Herramientas aplicadas

PyTorch, Hugging Face Transformers y Datasets, PEFT (LoRA), SHAP, scikit-learn, pandas/matplotlib y Google Colab (T4). Las dos herramientas que no se vieron a fondo en clase se detallan a continuación.

5.1 LoRA en detalle

La hipótesis de LoRA (Hu et al., 2021) es que la actualización ΔW que necesita una tarea al adaptar un modelo preentrenado tiene un *rango intrínseco* bajo. En lugar de entrenar $\Delta W \in \mathbb{R}^{d \times k}$ (d·k parámetros), se parametriza $\Delta W = B \cdot A$ y se entrenan solo r·(d+k) parámetros. B se inicializa en cero y A con una distribución gaussiana, de modo que ΔW = 0 al inicio y el modelo parte exactamente del preentrenado. El factor α/r reescala la actualización para que cambiar r no obligue a reajustar la tasa de aprendizaje. Al terminar, B·A puede fusionarse

en W (`merge_and_unload`), por lo que **no añade latencia en inferencia**. Con r = 8 sobre query/value de 12 capas se entrenan ≈0.89 M parámetros (0.32 % del modelo), que incluyen la cabeza de clasificación.

5.2 SHAP en detalle

SHAP asigna a cada token una contribución basada en los valores de Shapley de la teoría de juegos: el promedio de cuánto cambia la salida del modelo al incluir ese token frente a enmascararlo, sobre muchas coaliciones de tokens. Para texto se usa el *Partition explainer*, que agrupa jerárquicamente los tokens para aproximar los valores de Shapley sin evaluar todas las 2^n coaliciones. La suma de las contribuciones más el valor base reproduce la probabilidad de la clase. Una contribución positiva empuja la predicción hacia esa clase.

5.3 Uso de IA generativa

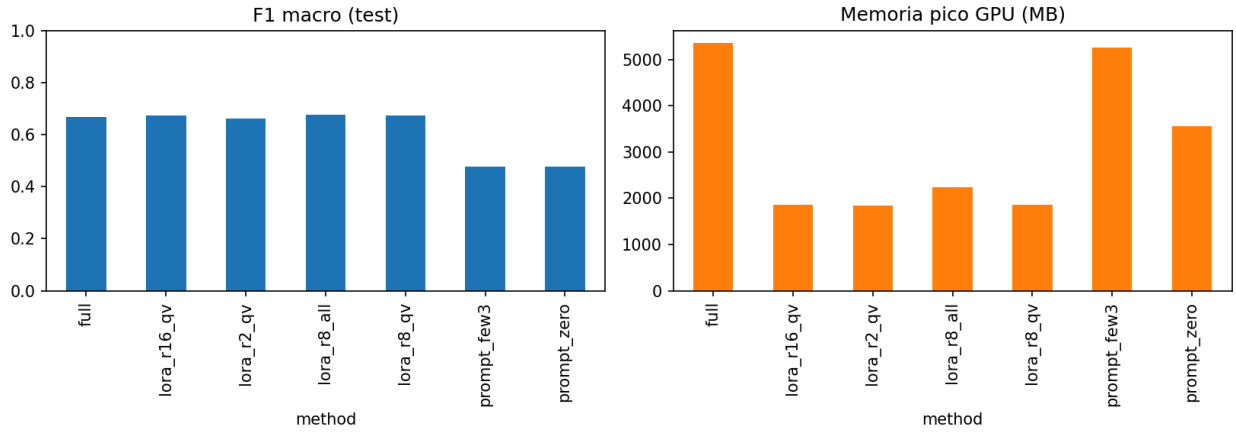
Se usó Claude Code para elegir los modelos, escribir el código y redactar borradores. El historial de prompts, los errores de la IA que se encontraron al ejecutar y cómo se validó cada respuesta están en el Anexo A.

6. Resultados

6.1 Comparación principal (tweets, test: 870 ejemplos)

Método	Semillas	Accuracy	F1 macro	Params entrenables	% del total	Memoria pico (MB)	Tiempo (s)
Prompting zero-shot (Qwen2.5-1.5B)	1	0.580	0.478	0	0	3,554	20.6 (inferencia)
Prompting few-shot, 3 por clase	3	0.571 ± 0.023	0.479 ± 0.026	0	0	5,254	93.1 (inferencia)
Fine-tuning completo (XLM-R base)	5	0.673 ± 0.008	0.667 ± 0.012	278,045,955	100	5,358	421.6
LoRA r=8, q/v (XLM-R base)	5	**0.676 ± 0.012**	**0.673 ± 0.013**	887,811	0.32	**1,853**	**32.8**

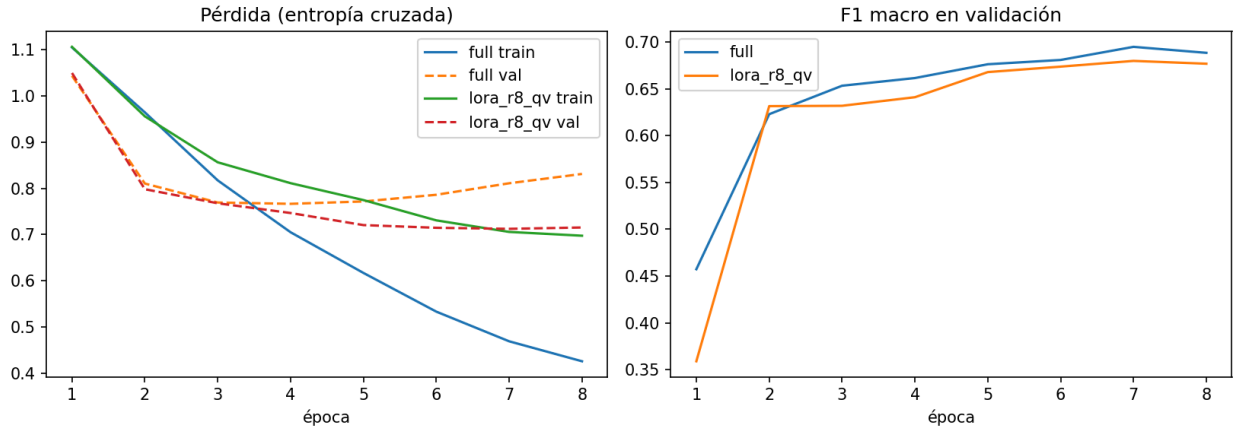
El zero-shot es determinista, por eso solo tiene una corrida. Los tiempos de prompting miden solo inferencia y no son comparables con los de entrenamiento.



F1 macro en test y memoria pico de GPU por método.

Lectura. LoRA iguala al fine-tuning completo: la diferencia de F1 (0.673 vs 0.667) es menor que una desviación estándar entre semillas, así que no hay evidencia de que alguno sea mejor. El costo, en cambio, difiere mucho: LoRA entrena **313 veces menos parámetros**, usa **2.9 veces menos memoria** y es **12.9 veces más rápido** en esta tarea. Parte del ahorro de tiempo se debe a que el fine-tuning completo también actualiza la matriz de *embeddings* (250 k tokens × 768 ≈ 192 M parámetros), que LoRA mantiene congelada. Ambos superan claramente al prompting (+19 puntos de F1).

6.2 Curvas de entrenamiento



Pérdida y F1 de validación por época (media de 5 semillas).

El fine-tuning completo sobreajusta: su pérdida de entrenamiento baja hasta ≈0.43 mientras la de validación llega a un mínimo en la época 4 (≈0.77) y luego sube (≈0.83). LoRA, con muchos menos grados de libertad, no muestra ese comportamiento y su pérdida de validación se estabiliza (≈0.71). En F1 de validación ambos terminan cerca (≈0.68–0.69). Los resultados de test usan el mejor checkpoint por F1 de validación, lo que reduce el efecto del sobreajuste del modelo completo. Es una regularización implícita: LoRA es más estable sin necesitar *early stopping* tan agresivo.

6.3 Ablación de LoRA (tweets, 3 semillas)

Configuración	Params entrenables	Accuracy	F1 macro	Memoria (MB)	Tiempo (s)
r=2, q/v	666,627	0.669 ± 0.014	0.663 ± 0.019	1,851	33.6

Configuración	Params entrenables	Accuracy	F1 macro	Memoria (MB)	Tiempo (s)
r=8, q/v (principal, 5 semillas)	887,811	0.676 ± 0.012	0.673 ± 0.013	1,853	32.8
r=16, q/v	1,182,723	0.678 ± 0.016	0.674 ± 0.017	1,859	34.1
r=8, todas las lineales	1,920,003	0.682 ± 0.010	0.678 ± 0.010	2,247	52.8

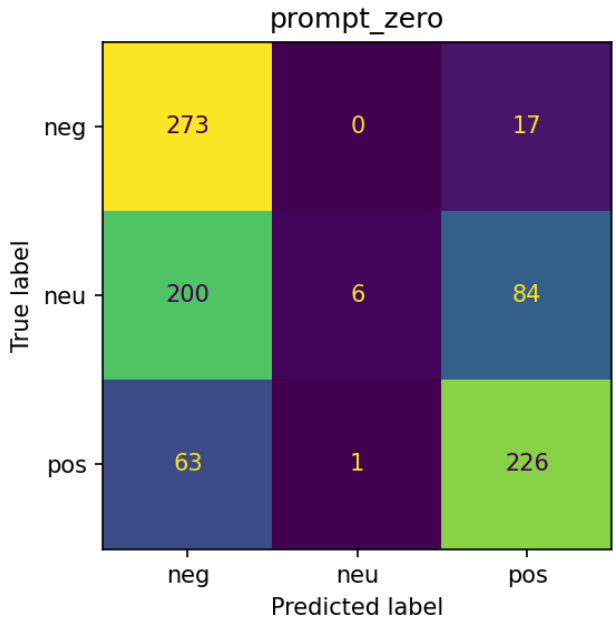
Las cuatro configuraciones quedan dentro de una desviación estándar entre sí. Subir el rango de 2 a 16 o adaptar más capas apenas modifica el F1 (≤1.5 puntos), mientras que aumenta parámetros y tiempo. El rango pequeño basta para esta tarea, y la ligera ventaja de «todas las lineales» no es concluyente con este tamaño de test.

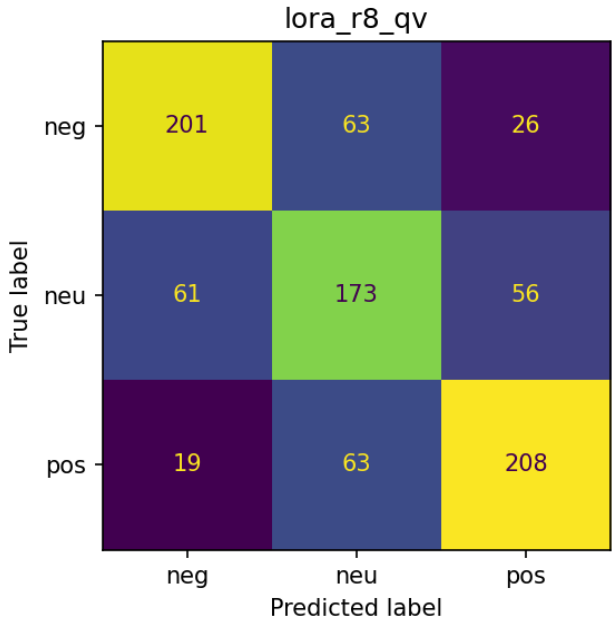
6.4 Segunda tarea: noticias en español (12 categorías, 3 semillas)

Método	Accuracy	F1 macro	Params entrenables	Memoria (MB)	Tiempo (s)
Fine-tuning completo	0.898 ± 0.004	0.897 ± 0.004	278,052,876	5,367	422.0
LoRA r=8, q/v	0.889 ± 0.006	0.886 ± 0.006	894,732	2,521	59.5

En noticias el fine-tuning completo es ligeramente superior (+1.1 puntos de F1, de 1.5 a 2 veces la desviación entre semillas), así que aquí LoRA sí pierde algo de calidad. Mantiene el patrón de costo: 311 veces menos parámetros, 2.1 veces menos memoria y 7.1 veces más rápido. La conclusión «LoRA iguala al modelo completo» se sostiene en tweets pero hay que matizarla: en una tarea con más clases y más datos, la diferencia es pequeña pero apreciable. (No se evaluó prompting en noticias porque la tarjeta del dataset no documenta los nombres de las categorías.)

6.5 Prompting y el problema de la clase «neutro»





El LLM casi nunca predice «neutro»: de 290 tweets neutros solo acierta 6 (recall 2 %), y 200 los clasifica como negativos. Con la clase neutra sin recuperar, el F1 macro queda en 0.48 aunque la accuracy sea 0.58. Los 3 ejemplos por clase no corrigen este sesgo (F1 0.479). El modelo ajustado con LoRA reparte los errores de forma mucho más equilibrada (recall 0.69 / 0.60 / 0.72 en negativo/neutro/positivo). Esto indica que el prompting sin calibración no captura el criterio de anotación de esta base de datos, y que el entrenamiento aporta, sobre todo, esa frontera de decisión.

6.6 Análisis de errores con SHAP (modelo LoRA, semilla 0)

De los 870 tweets de test, el modelo LoRA de la semilla 0 comete 288 errores (33.1 %; ver su matriz de confusión en 6.5); se explicaron 40 errores seleccionados al azar y 20 aciertos.

Real Predicho	negativo	neutro	positivo
negativo	–	11	4
neutro	6	–	9
positivo	2	8	–

1. La clase neutra concentra los errores. 34 de 40 errores involucran el neutro (15 reales neutros mal clasificados y 19 predichos como neutro). Solo 6 son confusiones directas negativo↔positivo. Es coherente con la matriz de confusión global y con que la frontera polar/neutro es subjetiva.

2. Palabras con valor de sentimiento explícito dominan la decisión, incluso cuando llevan al error. Las palabras que más empujan hacia la clase predicha en los errores son *gracias* (SHAP medio +0.27 con n=2; +0.29 en aciertos), *bueno* (+0.32), *mejor*, *mal*, el emoji ❤ y los signos de exclamación. Ejemplo: «`@user pf ya ves es la muerte muchas gracias bonita ❤❤❤`» es neutro para los anotadores, pero el modelo lo predice positivo; «gracias», «bonita» (+0.29) y ❤ están entre las palabras de mayor aporte en los errores. Algo similar ocurre con «`... gracias por apoyar al cine hecho en las regiones`».

3. Ironía y sarcasmo. «`Que bien, cuando marca el Madrid a mi portátil le apetece darme un pantallazo azul...`» (negativo) se predice positivo, y «bien» es la palabra con mayor aporte en los errores (+0.55); «`Es`

`gracioso cuando intentan sacarte del medio ... poniendo buena cara`» (negativo) también se predice positivo, con «buena» (+0.34) entre las palabras de mayor aporte. El modelo se apoya en el léxico y no en el contexto pragmático.

4. Hechos sin léxico emocional. «`@user @user fallecido hace meses. Hay q contrastar.`» o «`En 3 dias se acaba este largo camino de 8 años`» (negativos) se predicen neutros porque no contienen palabras de polaridad: la emoción es implícita.

5. Las palabras funcionales y las menciones no contribuyen (`@user`, «de», «la», «que», «a»: SHAP ≈ 0), lo que es una señal sana de que el modelo no usa artefactos del dataset como atajo. Hay que advertir que el tokenizador de XLM-R parte las palabras en subpalabras (p. ej. «gra» por «gracias», «bon» por «bonita»), por lo que algunas filas de `palabras_errores.csv` son fragmentos.

Limitación: solo 40 errores y 20 aciertos (SHAP con texto es costoso: ≈4 minutos para 60 ejemplos); la mayoría de las palabras aparecen una sola vez, así que el análisis es cualitativo. Los aportes por palabra citados son los de `results/explain/palabras_errores.csv`; no se revisó una a una cada visualización de `shap_text.html`.

7. Conclusión

- **LoRA ofrece calidad comparable a una fracción del costo.** En tweets, LoRA (F1 0.673 ± 0.013) y el fine-tuning completo (0.667 ± 0.012) son estadísticamente indistinguibles, con 313 veces menos parámetros entrenables, 2.9 veces menos memoria (1.9 vs 5.4 GB) y 12.9 veces menos tiempo. En noticias hay una pequeña pérdida (–1.1 puntos de F1) con el mismo patrón de ahorro.
- **El prompting sin entrenamiento no alcanza.** Con un LLM de 1.5 B parámetros el F1 macro es 0.48, 19 puntos por debajo, principalmente porque casi nunca predice «neutro». Pocos ejemplos en el prompt no lo corrigen.
- **Recomendación según recursos.** Sin GPU o sin datos etiquetados: prompting, pero esperar poca calidad en tareas con fronteras subjetivas. GPU pequeña (≈2 GB sobre el modelo): LoRA, que es la opción por defecto. GPU grande, mucho tiempo y datos abundantes: el fine-tuning completo puede ganar un punto adicional en tareas más complejas, pero hay que vigilar el sobreajuste.
- **Los errores restantes no son de capacidad de adaptación sino de la tarea:** ambigüedad del «neutro», ironía y emociones implícitas, que ningún método de adaptación aquí evaluado resuelve.
- **Limitaciones.** Test de 870 ejemplos; un solo modelo base y un solo diseño de prompt, sin calibrar; tasas de aprendizaje típicas sin búsqueda de hiperparámetros para ningún método; tiempos medidos en una sesión de Colab (pueden variar); SHAP sobre una muestra pequeña. Trabajo futuro: LoRA sobre un LLM generativo, calibración del prompting contra el sesgo hacia clases polares y más ejemplos de ironía.

8. Bibliografía

- Hu, E. J., et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685.
- Conneau, A., et al. (2020). *Unsupervised Cross-lingual Representation Learning at Scale* (XLM-R). ACL.
- Devlin, J., et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL.
- Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS.
- Loshchilov, I. & Hutter, F. (2019). *Decoupled Weight Decay Regularization* (AdamW). ICLR.
- Lundberg, S. & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions* (SHAP). NeurIPS.

- Barbieri, F., et al. (2022). *XLM-T: Multilingual Language Models in Twitter for Sentiment Analysis and Beyond*. LREC. (Dataset `cardiffnlp/tweet_sentiment_multilingual`.)
- Qwen Team (2024). *Qwen2.5 Technical Report*. arXiv:2412.15115.
- Mangrulkar, S., et al. *PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods*. Hugging Face, https://github.com/huggingface/peft.
- Dataset `mteb/SpanishNewsClassification`, Hugging Face (licencia MIT).

Anexo A. Uso de IA generativa

Herramienta: Claude Code (Anthropic). Detalle completo de prompts en `docs/prompts_ia.md`.

#	Prompt (resumen)	Resultado	Evaluación
1	Se adjuntó el enunciado, la propuesta y los datasets, y se pidió elegir el modelo para una T4 gratuita y completar el proyecto.	Propuso XLM-R base (full FT vs LoRA) y Qwen2.5-1.5B-Instruct (prompting); descartó el modelo de Cardiff por fuga de datos.	Satisfactorio: la justificación es verificable (la T4 no tiene bf16; 278 M parámetros caben en 16 GB; el modelo de Cardiff se entrenó con estos tweets).
2	Generar el código del experimento (fine-tuning, LoRA, prompting, SHAP, agregación y notebook).	Código modular en `src/`.	**Parcialmente incorrecto** al primer intento (ver errores). Se probó en CPU con un modelo diminuto antes de ir a Colab.
3	Se entregaron el notebook ejecutado y el zip de resultados para completar el informe.	Tablas, análisis e informe redactados con los números reales.	Se contrastaron las cifras del informe contra `tabla_comparativa.md` y contra las matrices de confusión; ver correcciones.

Errores de la IA detectados.

1. Supuso que `load_dataset("cardiffnlp/tweet_sentiment_multilingual")` funcionaría. El dataset usa un script de carga que `datasets ≥ 4` ya no soporta; se leyeron los `.jsonl` directamente tras consultar el script del repositorio.
2. Las etiquetas del `.jsonl` llegan como *strings* (`"0"`), lo que rompía el *collator*; se convirtieron a entero.
3. Usó `warmup_ratio`, argumento eliminado en transformers 5; se reemplazó por `warmup_steps`.
4. La ficha del dataset indica 1,838 / 323 / 869 ejemplos y la descarga trae 1,839 / 324 / 870.

Cómo se evaluaron las respuestas. Pruebas de humo en CPU, verificación de que los tokens iniciales de las etiquetas en Qwen fueran distintos, comparación contra la documentación de las librerías y, para el informe, contraste de cada cifra contra las salidas reales del experimento. Las métricas provienen de las corridas en Colab, no de la IA.

Aporte de la IA. Aceleró la elección de modelos y la escritura del código y los borradores. La corrida, la lectura crítica de resultados y las conclusiones fueron verificadas por el equipo.